@ codedsoul_05

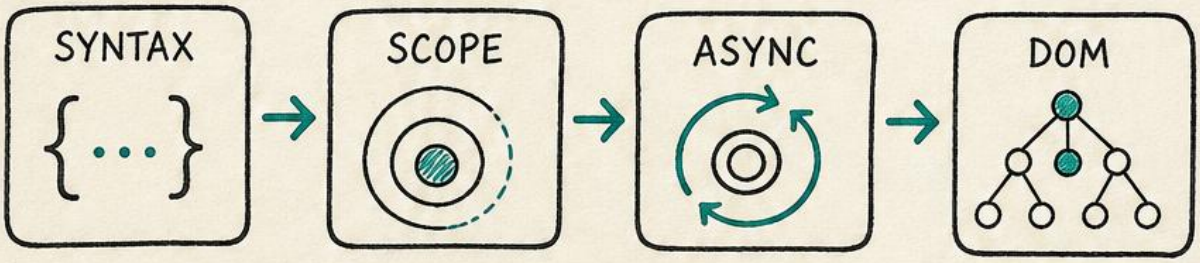
JS NOTES v2.0 ★

JAVASCRIPT

BEGINNER → ADVANCED

COMPLETE PREMIUM NOTES

From first console.log to async mastery — everything you need to write modern JavaScript with confidence.



"The language of the web — and beyond."

Syntax • Functions • Closures • DOM • Async • Modules • Patterns

1 WHAT'S INSIDE

1. Fundamentals
2. Types
3. Functions
4. Scope/Closures
5. Objects/this
6. Arrays
7. DOM
8. Async
9. Modules
10. Classes
11. Patterns
12. Tooling

2 BY THE NUMBERS

- 40+ Topics
- 80+ Snippets
- Event Loop Mastery
- Beginner to Pro
- Interview Ready
- Practice Recipes

3 TOPICS COVERED

- ☑ Basics
- ☑ Types & Coercion
- ☑ Scope & Closures
- ☑ this & Prototypes
- ☑ DOM & Events
- ☑ Promises & async/await
- ☑ ES Modules
- ☑ Fetch & Web APIs
- ☑ Patterns
- ☑ Node/npm

★ Scope. this. Event Loop. Async. ★

01 – ROADMAP & HOW JS RUNS

GOAL: Learn JavaScript core, then web APIs. Engine + host = where JS runs.

1 BEGINNER SYNTAX

variables, types, operators, control flow

JS

2 FUNCTIONS / OBJECTS

functions, scope, closures, this

f()

{ }

3 DOM

select, update, events, delegation

4 ASYNC

event loop, callbacks, promises, async/await

5 MODERN ES

destructuring, modules, optional chaining

export

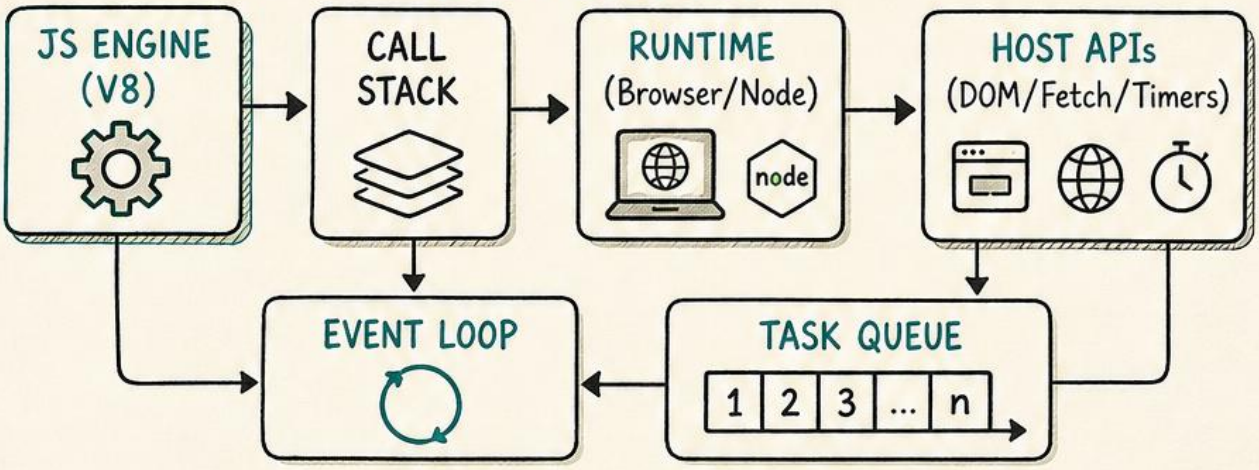
{ }

6 PATTERNS / TOOLING

patterns, errors, npm, bundlers

npm

HOW JS RUNS



WHERE TO PRACTICE

BROWSER CONSOLE

- Open DevTools (F12)
- Console tab
- Great for DOM & Web APIs
- Instant feedback

VS

NODE REPL

JS

- Run 'node' in terminal
- Test JS core features
- No DOM, but full JS power
- Great for backend logic

Language core ≠ Web APIs – learn both.

02 — BASICS: VARIABLES & OPERATORS

	let	const	var
Scope	Block	Block	Function (or Global)
Reassignment	Yes	No	Yes
Redeclaration in same scope	No	No	Yes
Hoisting / TDZ	TDZ until init	TDZ until init	Hoisted, init as undefined
Global object property	No	No	Yes (window)

NOTE: Prefer const, then let. Avoid var.

NAMING RULES

- Must start with a letter, _ or $
- Can contain letters, numbers, _ or $
- Case-sensitive
- Cannot be a reserved keyword

GOOD EXAMPLES

userName, _count, $price, total2

AVOID

2name, user-name, let, class

TEMPLATE LITERALS

- Use backticks ` `
- Embed expressions with ${}
- Supports multi-line strings

```
const name = "Alex";
const msg = `Hello ${name}!`
// Hello Alex!
```

OPTIONAL CHAINING (?.)

Safely access nested properties.

```
const user = { profile: { name: "Sam" } };
user.profile?.name;      // "Sam"
user.address?.city;      // undefined
user.settings?.theme?.mode; // undefined
```

OPERATORS CHEAT SHEET

ARITHMETIC

+ - * / %

** (exponent)

++ --

COMPARISON

== != > < >= <=

=== !==

Use === for equality

LOGICAL

&& (AND)

|| (OR)

?? (Nullish coalescing)

OTHER

! (NOT)

?. (Optional chaining)

?: (Ternary)

TRUTHY

- true
- "non-empty string"
- 1, -1, 3.14
- [], {}
- function() {}
- Infinity

FALSY

- false
- 0, -0
- "" (empty string)
- null
- undefined
- NaN

COMMENTS

```
// This is a single-line comment

/*
  This is a
  multi-line
  comment
*/
```

COMMON PITFALLS

- == vs ===
  == performs type coercion (can be surprising)

```
'5' == 5   // true
'5' === 5  // false
```

- const mutation
  const prevents reassignment, but allows mutation of objects/arrays

```
const obj = { x: 1 };
obj.x = 2;   // allowed ✓
obj = {};    // TypeError ✗
```

PREFER CONST, THEN LET. AVOID VAR.

03 — TYPES & COERCION

string	number	bigint	boolean	undefined	null	symbol
ex: "hello"	ex: 42	ex: 123n	ex: true	ex: undefined	ex: null	ex: Symbol()

typeof QUIRKS

- typeof null === 'object' ← historical bug!
- Arrays are objects
  typeof [] === 'object'
  [1, 2, 3] ↑ object
- Functions report 'function'
  typeof function() {} === 'function'

EVERYTHING HERE IS AN OBJECT

OBJECTS

{ a: 1, b: 'x' }

typeof {} === 'object'

ARRAYS

[1, 2, 3]

typeof [] === 'object'

FUNCTIONS

function f() {}

typeof f === 'function'

EXPLICIT CONVERSION (YOU DO IT)

Number(value)

Number('42') → 42
Number('') → 0
Number(true) → 1
Number('x') → NaN

NaN = Not a Number

String(value)

String(42) → '42'
String(true) → 'true'
String(null) → 'null'
String(undefined) → 'undefined'

Boolean(value)

Boolean(1) → true
Boolean(0) → false
Boolean('') → false
Boolean('hi') → true
Boolean(null) → false
Boolean([]) → true

IMPLICIT COERCION (PITFALLS!)

'5' + 2	→ '52'	string concat!
'5' - 2	→ 3	string to number
[] == 0	→ true	[] → '' → 0
null == undefined	→ true	loose equality

Avoid surprises. Use ===.

NaN

- NaN is not equal to anything, not even itself.

```
NaN === NaN   // false
```

Use Number.isNaN to be safe.

isNaN vs Number.isNaN

isNaN('x') → true ← coerces first
isNaN('123') → false

Number.isNaN('x') → false ← no coercion
Number.isNaN(NaN) → true

JSON: PARSE & STRINGIFY

JSON.parse (string → value)

```
const user = JSON.parse('{"id":1,"name":"A"}');
// { id: 1, name: 'A' }
```

JSON.stringify (value → string)

```
const json = JSON.stringify({ id: 1, name: 'A' });
// '{"id":1,"name":"A"}'
```

Know falsy values. Prefer ===.

04 — FUNCTIONS

1 DECLARATION vs EXPRESSION vs ARROW

```
// Declaration
function sum(a, b) {
  return a + b;
}
// Expression
const sumExp = function (a, b) {
  return a + b;
};
// Arrow
const sumArrow = (a, b) => a + b;
```

2 DEFAULTS AND REST (...args)

```
function greet(name = 'guest', age = 18) {
  return `Hi ${name}, age ${age}`;
}

function sumAll(...nums) {
  return nums.reduce((acc, n) => acc + n, 0);
}
```

→ ...packs remaining args into an array.

3 RETURN VALUES & EARLY RETURN

```
function getLabel(value) {
  if (value == null) return 'empty';
  if (typeof value === 'string') return 'text';
  if (Array.isArray(value)) return 'array';
  return 'other';
}
```

TIP: Early return keeps code flat and easier to read.

4 HIGHER-ORDER FUNCTIONS PASS & RETURN FUNCTIONS

```
function withLog(fn) {
  return function (...args) {
    console.log('start');
    const result = fn(...args);
    console.log('end');
    return result;
  };
}
const add = (a, b) => a + b;
const addWithLog = withLog(add);
addWithLog(2, 3); // logs, then returns 5
```

5 ARROW: NO OWN this / arguments

```
const obj = {
  count: 0,
  incLater() {
    setTimeout(() => {
      this.count++;     // refers to obj
      console.log(this.count);
    }, 500);
  }
}
obj.incLater(); // 0 → 1 after 500ms
```

Arrow functions don't have their own this or arguments — they inherit from the surrounding scope.

6 IIFE — IMMEDIATELY INVOKED FUNCTION EXPRESSION

```
(function () {
  const msg = 'I run now!';
  console.log(msg);
})();
```

Modern note: Use modules instead. IIFEs are rarely needed now.

7 CALLBACKS INTRO — ASYNC WITH setTimeout & fetchData EXAMPLE

```
function fetchData(callback) {
  console.log('Fetching data...');
  setTimeout(() => {
    const data = { id: 1, name: 'Ada Lovelace'};
    callback(null, data); // null = no error
  }, 1000);
}
```

→

```
fetchData((err, data) => {
  if (err) {
    console.error('Error:', err);
    return;
  }
  console.log('Data:', data);
});
```

Callback runs after async work is done.

Functions are values — treat them that way.

05 — SCOPE & CLOSURES

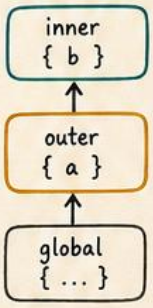
KEY PAGE

GOAL:
Understand how scope works in JavaScript and how closures let functions remember their environment.

1 Lexical scope

```
const outer = () => {
  const a = 'outer'
  const inner = () => {
    const b = 'inner'
    console.log(a, b)
  }
  inner()
}
outer()  // outer inner
```

Scope chain



- Inner looks up variables in outer, then outer's outer, and so on.

2 Block vs function scope

✅ let / const (block scoped)

```
{
  let x = 10
  const y = 20
  console.log(x)  // 10
}
console.log(x) // ReferenceError
```

↳ TDZ (Temporal Dead Zone) from start of block until declaration.

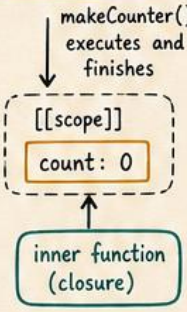
❌ var (function scoped)

```
{
  var x = 10
}
console.log(x)  // 10
```

- var is hoisted and accessible in the entire function.

3 Closures: remember the environment

```
function makeCounter() {
  let count = 0
  return function () {
    count++
    return count
  }
}
const counter = makeCounter()
counter()  // 1
counter()  // 2
counter()  // 3
```



- Outer function is gone, but the inner function remembers its variables.

4 Classic loop + var bug

❌ Using var (bug)

```
for (var i = 0; i < 3; i++) {
  setTimeout(() => {
    console.log(i)
  }, 0)
}
```

3 3 3

(after timeouts)

✅ Using let (fix)

```
for (let i = 0; i < 3; i++) {
  setTimeout(() => {
    console.log(i)
  }, 0)
}
```

0 1 2

(each iteration has its own i)

5 Module pattern / private state

via IIFE + closure

```
const CounterModule = (() => {
  let count = 0  // private
  return {
    inc() { count++; return count },
    get() { return count }
  }
})()
CounterModule.inc()  // 1
CounterModule.inc()  // 2
CounterModule.get()  // 2
```



- State is private but persists via closure.

6 Garbage collection & closures

Closures keep references to their environments.



- As long as the closure is reachable, the environment (and its data) stays alive.
- Once no references remain, GC can free it.

Closures = functions + remembered scope.

06 — OBJECTS & THIS

1. OBJECT LITERALS

key : value pairs

- **Dot vs Bracket**

```
obj.name        // dot
obj['name']     // bracket
```

- **Shorthand**

```
const name = 'JS'
const obj = { name }  // { name: name }
```

- **Computed Keys**

```
const key = 'id'
const obj = {[key]: 101, ['u'+'ser']: 'Alex'}
```

2. DESTRUCTURING OBJECTS

- **Rename**

```
const { name: n, age: a } = user
console.log(n, a)
```

- **Rest Params**

```
const { id, ...rest } = user
// rest has remaining properties
```



3. THIS BINDING RULES

• Default (strict mode)

```
this === undefined
```

fn()

• Implicit (object context)

```
const obj = {
  name: 'JS',
  show() {
    console.log(this)
  }
}
obj.show()
```

this → obj

• Explicit (call / apply / bind)

```
function show() {
  console.log(this)
}
const obj = { name: 'JS' }
show.call(obj)
show.apply(obj, [])
const bound = show.bind(obj)
bound()
```

• new (constructor)

```
function Person(name) {
  this.name = name
}
const p = new Person('JS')
```

p

this → new instance

• Arrow (lexical this)

```
const obj = {
  name: 'JS',
  show: () => {
    console.log(this)
  }
}
obj.show()
```

this → outer scope

4. OPTIONAL CHAINING ?. AND NULLISH ??

```
const user = { profile: { name: 'Alex'}}
user?.profile?.name     // 'Alex'
user?.address?.city     // undefined
```

```
const name = user.name ?? 'Guest'
   // uses default only if null or undefined
```

5. OBJECT.KEYS / VALUES / ENTRIES

```
const obj = { a: 1, b: 2, c: 3 }
Object.keys(obj)     // ['a','b','c']
Object.values(obj)   // [1,2,3]
Object.entries(obj)  // [['a',1],['b',2],['c',3]]
```

6. SPREAD & SHALLOW CLONE PITFALL

```
const original = {
  name: 'JS',
  address: { city: 'NY' }
}
const clone = { ...original }     // shallow clone
clone.address.city = 'LA'
console.log(original.address.city) // 'LA'
```

Nested objects are still shared!

7. LOSING THIS IN CALLBACKS — BIND VS ARROW FIX

```
const obj = {
  name: 'JS',
  log() {
    setTimeout(function() {
      console.log(this.name)
    }, 100)
  }
}
```

Problem: this → window/undefined

Fix 1: bind

```
setTimeout(function() {
  console.log(this.name)
}.bind(this), 100)
```

Fix 2: arrow

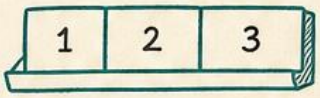
```
setTimeout(() => {
  console.log(this.name)
}, 100)
```

this depends on how the function is called.

07 — ARRAYS & METHODS

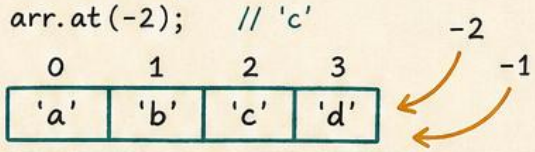
1 Create arrays

```
const a = [1, 2, 3];
const b = new Array(1, 2, 3);
const c = Array.of(1, 2, 3);
const d = Array.from('hi');
```



2 Index, length, at(-1)

```
const arr = ['a', 'b', 'c', 'd'];
arr[0];      // 'a'
arr[arr.length - 1]; // 'd'
arr.at(-1);    // 'd'
arr.at(-2);    // 'c'
```



3 Mutating methods

- push() // add to end
- pop() // remove from end
- shift() // remove from start
- unshift() // add to start
- splice() // add/remove at index
- sort() // sort in place
- reverse() // reverse in place

changes origina!

4 Non-mutating methods

- map() // transform
- filter() // keep some
- reduce() // accumulate
- find() // first match
- some() // any match?
- every() // all match?
- includes() // contains?
- slice() // shallow copy
- concat() // merge arrays

returns new array

5 Spread & destructuring

```
// spread
const a = [1, 2];
const b = [...a, 3, 4]; // [1, 2, 3, 4]
const c = ['x', ...a, 'y'];
// destructuring
const [first, second, ...rest] = a;
// first=1, second=2, rest=[]
```

6 for...of vs forEach

```
// for...of (works with break/continue)
for (const item of arr) {
  if (item > 10) break;
}
// forEach (no break/continue)
arr.forEach((item) => {
  // can't break
});
```

early exit possible

runs all items

7 sort() compare pitfalls

```
// ✕ lexicographic (default)
[2, 10, 1].sort();
// [1, 10, 2] ← wrong for numbers
// ✓ numeric sort
[2, 10, 1].sort((a, b) => a - b);
// [1, 2, 10] ← correct
```

Always compare!

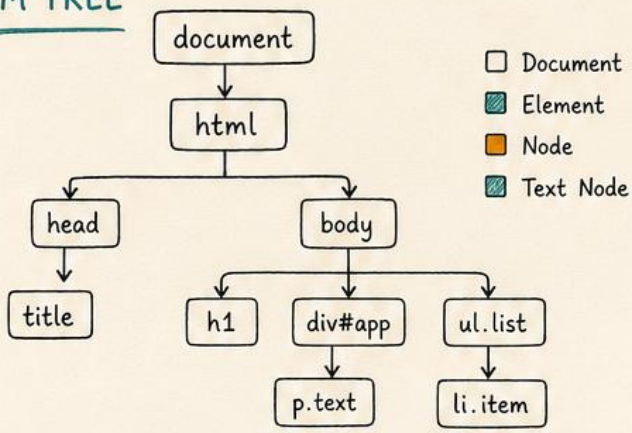
8 Quick recipes

- unique
  `[...new Set(arr)]` ← remove duplicates
- flat
  `arr.flat(Infinity)` ← flatten nested arrays
- join / split
  `arr.join(', ')` ← array ↔ string
  `str.split(',')`

Prefer map/filter/reduce for transforms.

08 — DOM & EVENTS

DOM TREE



SELECTING ELEMENTS

querySelector

→ first match

```
document.querySelector('.item')
```

querySelectorAll

→ all matches (NodeList)

```
document.querySelectorAll('li')
```

CREATE & INSERT

createElement

```
const el = document.createElement('div')
```

appendChild

```
parent.appendChild(el)
```



TEXT vs HTML

textContent

→ safe, shows as text

```
el.textContent = '<b>Hi</b>'
```

innerHTML

→ parses HTML (danger!)

```
el.innerHTML = '<b>Hi</b>'
```

⚠ XSS RISK
Do not use with user input!

CLASS LIST

- el.classList.add('active')
- el.classList.remove('active')
- el.classList.toggle('active')
- el.classList.contains('active')

```
class="btn active"
```

toggle me!

DATASET

Access data-* attributes

```
<div data-id="42" data-user="sam">
```

```
el.dataset.id      // "42"
el.dataset.user    // "sam"
```

All values are strings!

STYLES

```
el.style.color = 'teal'
el.style.background = '#f5a623'
```

```
el.style.cssText =
'padding: 8px; border: 1px solid
#ccc; border-radius: 6px;'
```

EVENTS

BASIC EVENTS

- click → button, link
- input → typing in field
- submit → form submit

```
btn.addEventListener('click', (e) => {
  console.log('clicked!', e.target);
});
```

EVENT CONTROL

preventDefault()

→ stop default action

```
e.preventDefault();
```

stopPropagation()

→ stop bubbling further up

```
e.stopPropagation();
```

EVENT DELEGATION

Listen on parent, handle children

```
<ul id="list">
  <li>One</li>
  <li>Two</li>
  <li>Three</li>
</ul>
```

Only 1 listener on parent!

```
document.getElementById('list')
  .addEventListener('click', (e) => {
    if (e.target.tagName === 'LI') {
      console.log('clicked', e.target.textContent);
    }
});
```

FORMDATA (Easy form data)

```
<form id="f">
  <input name="name" value="Sam">
  <input name="age" value="21">
  <button>Send</button>
</form>
```

→

```
const form = document.getElementById('f');
form.addEventListener('submit', (e) => {
  e.preventDefault();
  const data = new FormData(form);
  for (const [key, value] of data.entries()) {
    console.log(key, value);
  }
});
```

Output in console:

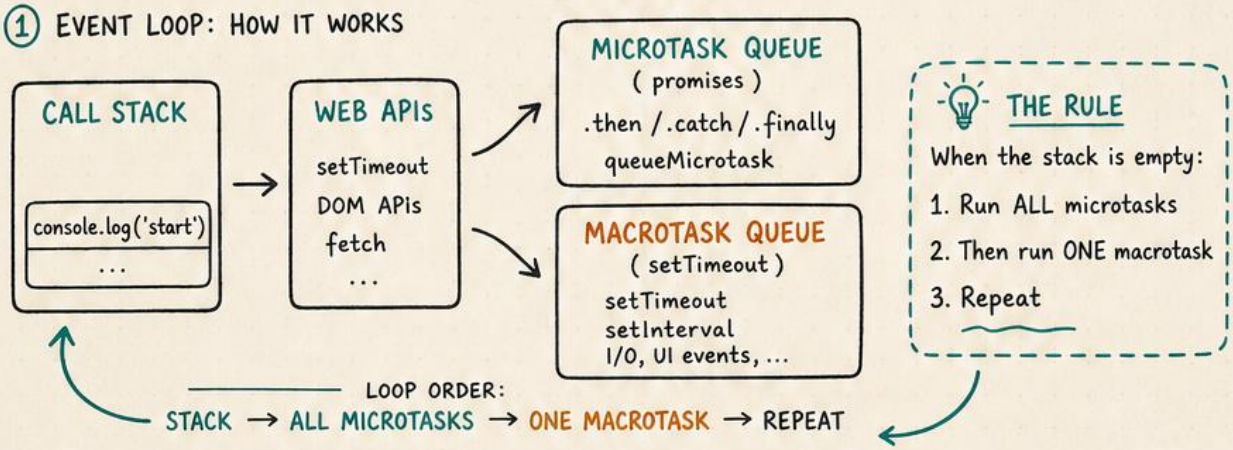
```
name  Sam
age   21
```

Never trust innerHTML with user input.

09 — ASYNC: EVENT LOOP & PROMISES

★ KEY PAGE ←

① EVENT LOOP: HOW IT WORKS



② CODE ORDER DEMO

```
console.log('start');
Promise.resolve().then(() =>
   console.log('promise'));
setTimeout(() =>
   console.log('timeout'), 0);
console.log('end');
```

OUTPUT ORDER

1. start
2. end
3. promise
4. timeout

③ PROMISE: BASICS

CREATE

```
const p = new Promise((resolve, reject) => {
   // async work
   resolve('ok');  // or reject(err);
});
```

STATES

PENDING (initial) → FULFILLED (resolved) → REJECTED (rejected)

④ .THEN • .CATCH • .FINALLY (CHAINING)

```
doSomething()
  .then(result => {
     return step2(result);
  }
  .then(res2 => {
     return step3(res2);
  })
  .catch(err => {
     handleError(err);
  })
  .finally(() => {
     cleanup();
  });
```

- `.then` → Runs when promise is fulfilled.
- `.catch` → Runs when promise is rejected.
- `.finally` → Runs always, either way.

⑤ HANDY PROMISE HELPERS

- `Promise.all([p1, p2, p3])` → Resolves when ALL fulfill, or rejects if any rejects.
- `Promise.allSettled([p1, p2])` → Waits for ALL to settle. Always resolves with their results.
- `Promise.race([p1, p2, p3])` → Settles as soon as the first one settles.
- `Promise.any([p1, p2, p3])` → Resolves as soon as any one fulfills. Rejects if all reject.

⑥ CALLBACK HELL vs PROMISE CHAIN

CALLBACK HELL

```
step1((err, r1) => {
  if (err) return handle(err);
  step2(r1, (err, r2) => {
    if (err) return handle(err);
    step3(r2, (err, r3) => {
      if (err) return handle(err);
      step4(r3, (err, r4) => {
        if (err) return handle(err);
        done(r4);
      });
    });
  });
});
```

⇒

PROMISE CHAIN

```
step1()
  .then(r1 => step2(r1))
  .then(r2 => step3(r2))
  .then(r3 => step4(r3))
  .then(r4 => done(r4))
  .catch(handle);
```

Clean, Readable, Maintainable!

★ Microtasks run before the next macrotask. ★

10 — ASYNC/AWAIT & FETCH

1 async/await basics



- async returns Promise
- await unwraps

```
async function getData() {
  const res = await fetch('/api/data');
  const data = await res.json();
  return data;
}

getData().then(console.log);
```

2 try/catch error handling



```
async function load() {
  try {
    const res = await fetch('/api/data');
    if (!res.ok) throw new Error('HTTP ' + res.status);
    const data = await res.json();
    return data;
  } catch (err) {
    console.error('Load failed:', err.message);
    throw err;
  }
}
```

3 fetch GET JSON with res.ok check



```
async function getJSON(url) {
  const res = await fetch(url);
  if (!res.ok) {
    throw new Error(`HTTP ${res.status}`);
  }
  return await res.json();
}

// usage
const data = await getJSON('/api/users');
```

4 fetch POST JSON Content-Type



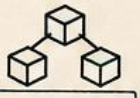
```
async function postJSON(url, payload) {
  const res = await fetch(url, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(payload)
  });
  if (!res.ok) throw new Error(`HTTP ${res.status}`);
  return await res.json();
}
// usage
await postJSON('/api/users', { name: 'Alice' });
```

5 AbortController timeout cancel



```
async function fetchWithTimeout(url, ms = 5000) {
  const controller = new AbortController();
  const id = setTimeout(() => controller.abort(), ms);
  try {
    const res = await fetch(url, { signal: controller.signal });
    if (!res.ok) throw new Error(`HTTP ${res.status}`);
    return await res.json();
  } catch (err) {
    if (err.name === 'AbortError') {
      throw new Error('Request timed out');
    }
    throw err;
  } finally {
    clearTimeout(id);
  }
}
```

6 Promise.all parallel requests



```
async function loadAll() {
  const urls = ['/api/a', '/api/b', '/api/c'];
  const promises = urls.map(u => fetch(u)
    .then(r => {
      if (!r.ok) throw new Error(`HTTP ${r.status}`);
      return r.json();
    })
  );
  const [a, b, c] = await Promise.all(promises);
  return { a, b, c };
}

// runs all requests in parallel
```

7 top-level await in modules




```
// in an ES module (.mjs or type: "module")
const res = await fetch('/api/config');
if (!res.ok) throw new Error(`HTTP ${res.status}`);
const config = await res.json();
export const appConfig = config;

// top-level await is allowed in modules
```

8 Pitfalls



- ✗ Forgetting await → you'll get a Promise, not the data.
- ✗ Not checking res.ok → can hide HTTP errors (4xx/5xx).
- ✗ Sequential awaits = slower
  Use Promise.all for independent requests.
- ✗ Swallowing errors → makes debugging hard.
  Always handle or rethrow.



await unwraps. Always handle HTTP errors.



11 — MODERN ES FEATURES

1 DEEP DESTRUCTURING

```
const user = {
  id: 1,
  profile: {
    name: 'Alex',
    socials: { twitter: '@x' }
  }
};
const { profile: {
  name,
  socials: { twitter }
} } = user;
// name: 'Alex', twitter: '@x'
```

2 REST / SPREAD

```
const arr = [1, 2, 3];
const [...copy, 4] = arr;
// [1,2,3,4]

const obj = { a: 1, b: 2 };
const newObj = { ...obj, c: 3 };
// { a:1, b:2, c:3 }

function sum(...nums) {
 return nums.reduce((a, b) => a + b, 0);
}
```

3 ENHANCED OBJECT LITERALS

```
const name = 'Sam';
const age = 25;
const getInfo = () => 'ok';

const obj = {
  name,
  age,
  getInfo,
  ['createdAt']: new Date()
};

// Shorthand, methods, computed
```

4 FOR...OF / FOR AWAIT...OF

```
for (const n of [10, 20, 30]) {
  console.log(n);
}

async function* gen() {
  yield 1; yield 2;
}
for await (const v of gen()) {
  console.log(v);
}
```

5 MAP SET WEAKMAP WEAKSET

```
const map = new Map();
map.set('a', 1).get('a'); // 1

const set = new Set([1, 2, 2]);
set.has(2); // true

const wm = new WeakMap();
const ws = new WeakSet();

// Unique, ordered, GC-friendly
```

6 SYMBOL BIGINT

```
const id = Symbol('id');
const obj = { [id]: 123 };

obj[id]; // 123

const big = 12345678901234567890n;

typeof big; // 'bigint'
```

7 OPTIONAL CHAINING ?. NULLISH ??

```
const user = { profile: null };
const name = user?.profile?.name;
// undefined

const v = null ?? 'default';
// 'default'
```

8 ARRAY FLAT / FLATMAP OBJECT.FROMENTRIES

```
const arr = [1, [2, [3, 4]]].flat(2);
// [1, 2, 3, 4]
const doubled = [1,2,3].flatMap(n => [n,n*2]);
// [1, 2, 2, 4, 3, 6]

const obj = Object.fromEntries(
  [['a', 1], ['b', 2]]
); // { a: 1, b:2}
```

9 STRING INCLUDES PADSTART REPLACEALL

```
'hello world'.includes('world');
// true

'7'.padStart(3, '0'); // '007'

'a-b-c'.replaceAll('-', '/');
// 'a/b/c'
```

10 MODULES (PREVIEW)

```
math.js
 export const add = (a, b) => a + b;
 export default function mul(a, b) {
  return a * b;
}
```

```
app.js
import mul, { add } from './math.js';
add(2,3); // 5
mul(2,3); // 6
```

11 PRIVATE FIELDS

```
class Person {
  #name;
  constructor(name) {
    this.#name = name;
  }
  getName() {
    return this.#name;
  }
}
const p = new Person('Alex');
p.getName(); // 'Alex'
// p.#name; // SyntaxError
```

★ PRIVATE BY DESIGN

- Not accessible outside class
- Not on instances
- Cleaner APIs

Write modern JS by default.

12 — MODULES (ESM)

1 export / export default / named exports

```
// math.js
export const PI = 3.14159;
export function add(a, b) {
  return a + b;
}

const multiply = (a, b) => a * b;
export default multiply;
```

2 named imports & import * as

```
// main.js
import { PI, add } from './math.js';
import * as MathUtils from './math.js';

console.log(PI, add(2, 3));
console.log(MathUtils.default(2, 3));
```

import * as gives you all exports as a namespace object.

3 type=module in browser; package.json "type": "module"

In HTML:

```
<script type="module" src="main.js"></script>
```

In package.json:

```
{
  "type": "module"
}
```

→ Treats .js files as ES modules by default.

4 circular dependency caution

```
a.js                          b.js
import { foo }                import { bar }
from './b.js'                 from './a.js'
export const foo              export const bar
```

Can lead to undefined bindings if used before initialized.

5 dynamic import() lazy load

```
button.addEventListener('click', async () => {
  const { heavyTask } = await import('./heavy.js');
  heavyTask();
});
```

Loads the module only when needed. Great for code-splitting!

6 vs CommonJS (CJS) interop note

```
ESM → CJS
import cjsModule from './cjs.cjs';

CJS → ESM (from CJS)
const esm = await import('./esm.mjs');
```

Interop works, but default/named mappings can be tricky.

7 tree-shaking idea unused exports removed

utils.js

```
export function used() {}
export function unused1() {}
export function unused2() {}
```

build / bundler (tree-shaking) →

bundle.js

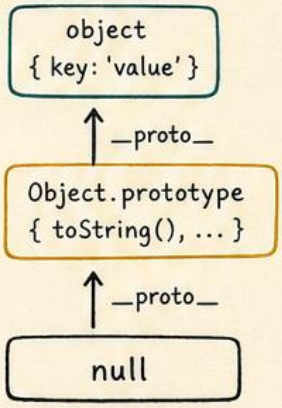
```
function used() {}
```

unused1, unused2 removed!

★ One module = one file = clear exports. ★

13 — PROTOTYPES & CLASSES

1 PROTOTYPE CHAIN



End of the chain. Top of the world.

2 CLASSES ARE SYNTAX SUGAR OVER PROTOTYPES

CLASS SYNTAX

```
class User {
  constructor(name) {
    this.name = name;
  }
  greet() {
    return `Hi ${this.name}`;
  }
}
```

→

UNDER THE HOOD

```
function User(name) {
  this.name = name;
}
User.prototype.greet =
function() {
  return 'Hi ' + this.name;
};
```

Same behavior, different clothes.

3 CONSTRUCTOR, METHODS, EXTENDS & SUPER

```
class Animal {
  constructor(name) {
    this.name = name;
  }
  speak() {
    return '...';
  }
}
class Dog extends Animal {
  constructor(name, breed) {
    super(name);   // call parent
    this.breed = breed;
  }
  speak() {
    return this.name + ` barks!`;
  }
}
```

`constructor()` runs on `new`

`extends` inherits from another class

`super()` calls parent constructor

`super.method()` calls parent method

4 STATIC METHODS

```
class MathUtil {
  static add(a, b) {
    return a + b;
  }
}

MathUtil.add(2, 3);   // 5

✗ const m = new MathUtil();
✗ m.add(2, 3);   // TypeError
```

Called on the class, not on instances.

Useful for utility functions.

5 PRIVATE FIELDS, GETTERS & SETTERS

```
class Account {
  #balance = 0;          // private
  constructor(owner) {
    this.owner = owner;
  }
  get balance() {        // getter
    return this.#balance;
  }
  set balance(val) {     // setter
    if (val < 0) throw new Error('Invalid');
    this.#balance = val;
  }
  deposit(amount) {
    this.#balance += amount;
  }
}
```

`#` makes it truly private.

Access via getters/setters.

6 INSTANCEOF & Object.create()

instanceof

```
class Person {}
const p = new Person();
p instanceof Person;     // true
p instanceof Object;     // true
p instanceof Array;      // false
```

✓ Checks if object is instance of a constructor (in the chain).

Object.create()

```
const proto = { greet() { return 'hi'; }};
const obj = Object.create(proto);
obj.greet(); // 'hi'
obj.__proto__ === proto; // true
```

Creates object with given prototype.

7 WHEN TO USE CLASS vs FUNCTIONS / CLOSURES

USE	CLASS	FUNCTIONS / CLOSURES
Modeling entities	Great for entities with state & methods	Possible, but more manual
Inheritance needed	Built-in extends / super	Use prototypes manually
Many instances	Methods on prototype (memory efficient)	Careful: methods per instance unless on prototype
Encapsulation	Private fields #, getters/setters	Use closures for true privacy
Lightweight / Functional	More boilerplate	Closures shine (simple, flexible)
Utility / Stateless	Use static methods	Or plain functions

★ Classes are sugar over prototypes — know both. ★

14 — ERRORS & DEBUGGING

SyntaxError



- Thrown when code violates JS grammar.
- Caught at parse time (before execution).

Ex.

```
if (x === {
          ^
```

ReferenceError



- Accessing a variable that is not defined.
- Happens at runtime.

Ex.

```
console.log(y);
    ^
```

TypeError



- Operation on a value of the wrong type.
- Common with null, undefined, or non-functions.

Ex.

```
null.foo();
      ^
```

throw / custom Error



- Use throw to raise errors manually.
- Create custom errors by extending Error.

Ex.

```
throw new Error(
  'Bad input');
```

try catch finally



- try: risky code
- catch: handle error
- finally: always runs

Ex.

```
try { ... }
catch (e) { ... }
finally { ... }
```

console methods

console.log()	info
console.table()	tabular
console.time()	start timer
console.timeEnd()	stop timer
console.assert()	test

log	output info	ⓘ
table	tabular data	▦
time	measure perf	◷
assert	assert condition	✓

debugger



- debugger statement pauses execution.
- Works with DevTools to inspect state.

Ex.

```
function foo() {
  debugger;
  ...
}
```

DevTools

- Inspect, debug, profile.

Breakpoints

- Pause on line, condition, XHR.

Network

- Inspect requests, responses, timing.

Sources & Maps

- Read original src with source maps.

use strict



- Enables stricter parsing and error handling.
- Helps catch silent errors and bad practices.

Ex.

```
'use strict';
// or at top of file
```

TOOLS



debugger



breakpoints



network tab



source maps



console



strict mode

Reproduce → isolate → fix → regression test.

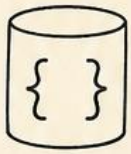
15 — STORAGE & WEB APIs

① WEB STORAGE & COOKIES — COMPARISON

FEATURE	localStorage	sessionStorage	Cookies
Scope	Per origin (scheme + domain + port)	Per origin + per tab	Per domain (configurable)
Lifetime	No expiry (until cleared)	Until tab/window is closed	Expires / Max-Age (you define)
Size (approx.)	5–10 MB	5–10 MB	4 KB per cookie ~ 20–50 cookies
Sent with Requests	No	No	Yes (if not HttpOnly & matches rules)
Use For	App data, prefs, cache, offline	Tab-specific data, form state, flow	Sessions, auth, server-side usage

② STORE OBJECTS? USE JSON

```
localStorage.setItem('user', JSON.stringify(userObj));
const user = JSON.parse(localStorage.getItem('user') || 'null');
```



Storage stores strings only!

③ URLSearchParams & HISTORY API

- Read & write query strings easily.
- history.pushState() / replaceState() to update URL without reload.

```
const params = new URLSearchParams(location.search);
params.get('q');
history.pushState({ page: 2 }, '', '?page=2');
```

④ INTERSECTION OBSERVER

- Observe visibility of elements in the viewport (lazy load, etc).

```
const io = new IntersectionObserver((entries) => {
  entries.forEach(e => { if (e.isIntersecting) { /* … */ } });
}, { threshold: 0.1 });
io.observe(targetEl);
```

⑤ RESIZE OBSERVER

- Observe size changes of an element.

```
const ro = new ResizeObserver(entries => {
  for (const entry of entries) { /* entry.contentRect */ }
});
ro.observe(targetEl);
```

⑥ TIMERS

- setTimeout(fn, delay)
- setInterval(fn, delay)
- clearTimeout(id) / clearInterval(id)

```
const tid = setTimeout(() => { /* … */ }, 1000);
const iid = setInterval(() => { /* … */ }, 2000);
clearTimeout(tid); clearInterval(iid);
```

⑦ REQUEST ANIMATION FRAME

- Smooth animations synced with the browser's repaint.

```
function animate(ts) {
  // update
  requestAnimationFrame(animate);
}
requestAnimationFrame(animate);
```

⑧ CLIPBOARD API

- Write: navigator.clipboard.writeText(text)
- Read : navigator.clipboard.readText()
- Requires secure context (HTTPS).

```
await navigator.clipboard.writeText('Hello!');
const text = await navigator.clipboard.readText();
```

⑨ GEOLOCATION API

- navigator.geolocation.getCurrentPosition()
- getCurrentPosition(success, error, options)
- Requires user permission.

```
navigator.geolocation.getCurrentPosition(
  pos => { console.log(pos.coords); },
  err => { console.error(err); },
  { enableHighAccuracy: true }
);
```

Permission is required. Always handle denied state.

Storage is sync & small — don't abuse it.

16 — USEFUL PATTERNS

1 Debounce / Throttle

```
// debounce
const debounce = (fn, wait = 300) => {
  let t;
  return (...args) => {
    clearTimeout(t);
    t = setTimeout(() => fn(...args), wait);
  };;
};
// throttle
const throttle = (fn, wait = 300) => {
  let last = 0;
  return (...args) => {
    const now = Date.now();
    if (now - last >= wait) {
      last = now; fn(...args);
    }
  }
};
```

2 Revealing module pattern

```
const counter = (() => {
  let count = 0;
  return {
    inc() { count++; return count; },
    dec() { count--; return count; },
    get() { return count; },
    reset() { count = 0; }
  };
})();

counter.inc(); // 1
counter.get(); // 1
```

3 Mini Event Emitter

```
const Emitter = () => {
  const events = {};
  return {
    on(evt, fn) { (events[evt] ||= []).push(fn); },
    off(evt, fn) {
      events[evt] = (events[evt] || []).filter(f => f!== fn);
    },
    emit(evt, ...args) {
      (events[evt] || []).forEach(fn => fn(...args));
    }
  };
};
const bus = Emitter();
bus.on('ping', x => console.log('pong', x));
bus.emit('ping', 42);
```

4 Immutable update with spread

```
const state = {
  user: {
    id: 1,
    name: 'Sam',
    prefs: { theme: 'dark' }
  }
};
const next = {
  ...state,
  user: {
    ...state.user,
    prefs: { ...state.user.prefs, theme: 'light' }
  }
};
```

5 Guard clauses early return

```
function getUser(id) {
  if (!id) return null;
  if (typeof id !== 'number') return null;
  if (id <= 0) return null;

  // happy path
  return db.users.find(u => u.id === id) || null;
}
```

6 Async pool / sequential map

```
async function mapSeries(arr, fn) {
  const out = [];
  for (const item of arr) {
    out.push(await fn(item));
  }
  return out;
}
```

7 Retry with backoff

```
async function retry(fn, tries = 3, base = 300) {
  let attempt = 0;
  while (true) {
    try { return await fn(); }
    catch (err) {
      attempt++;
      if (attempt >= tries) throw err;
      const wait = base * 2 ** (attempt - 1);
      await new Promise(r => setTimeout(r, wait));
    }
  }
}
```

8 Feature detect before use

```
function canShare() {
  return typeof navigator !== 'undefined' &&
         'share' in navigator;
}
if (canShare()) {
  navigator.share({ title: 'JS Notes', text: 'Hi!' });
} else {
  // fallback
  console.log('Share not supported');
}
```

Debounce input. Throttle scroll. Guard early.

17 — NODE, NPM & TOOLING

1 Node.js outside browser

Built-in modules for file, network, paths, and more.

```
// fs
import { readFileSync } from 'fs';
const data = readFileSync('a.txt', 'utf8');

// http
import http from 'http';
http.createServer((req, res) => {
  res.end('Hello');
}).listen(3000);

// path
import path from 'path';
const full = path.join(__dirname, 'a.txt');
```

2 npm basics

- package.json — project manifest
- scripts — automate tasks
- dependencies vs devDependencies

```
{
  "name": "my-app",
  "version": "1.0.0",
  "scripts": {
    "dev": "node index.js",
    "build": "vite build",
    "test": "vitest"
  },
  "dependencies": { "express": "^4.18.2" },
  "devDependencies": { "vite": "^5.2.0" }
}
```

3 npx — run without global install

npx runs packages from npm without installing them globally.

```
npx cowsay hello
```

```
npx vite        # create / run
npx eslint .    # lint code
```

4 SemVer — version ranges

^ major compatible
^1.2.3 => >=1.2.3 <2.0.0

~ minor compatible
~1.2.3 => >=1.2.3 <1.3.0

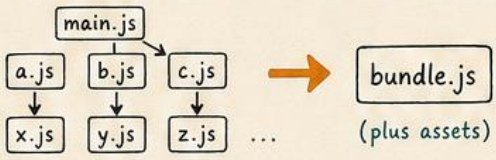
no symbol — exact version
1.2.3 => =1.2.3

newer ↑ older

5 Bundlers — idea

Bundle many modules → fewer files for browser.

Before (many files) → After (bundled)



Examples: Vite, Webpack, Rollup, esbuild

6 ESLint + Prettier

ESLint	Prettier
✓ finds issues	✓ formats code
✓ detects bugs	✓ consistent style
✓ enforces style	✓ saves time

Work great together!
Lint for correctness, format for consistency. ♡

7 TypeScript — quick intro

- Superset of JavaScript
- Static types
- Better tooling & DX

```
function add(a: number, b: number): number {
  return a + b;
}
const sum = add(2, 3);  // number
```

★ package.json is the project manifest. ★

18 — COMMAND CHEAT SHEET

Your daily JS tools. Copy. Run. Ship.

ARRAY METHODS

```
arr.map(fn)              // transform
arr.filter(fn)           // filter
arr.reduce(fn, init)     // reduce to one
arr.find(fn)             // first match
arr.findIndex(fn)        // index of first
arr.some(fn)             // any true?
arr.every(fn)            // all true?
arr.includes(x)          // contains?
arr.push(x)              // add to end
arr.pop()                // remove last
arr.shift()              // remove first
arr.unshift(x)           // add to start
arr.slice(a, b)          // shallow copy
arr.splice(i, n, ...items) // add/remove
arr.sort((a,b) => a-b)   // sort
arr.reverse()            // reverse
```

PROMISE HELPERS

```
Promise.resolve(val)     // already resolved
Promise.reject(err)      // already rejected
Promise.all([p1, p2])    // all succeed or fail
Promise.allSettled(ps)   // all results
Promise.race(ps)         // first settle wins
Promise.any(ps)          // first fulfilled
```

```
Promise.all([fetch('/a'), fetch('/b')])
  .then(([a,b]) => { /* both ok */ })
  .catch(err => { /* any failed */ })
```

DESTRUCTURING

```
// Array
const [a, b, ...rest] = arr;

// Object
const { x, y, z = 0, ...others } = obj;

// Nested
const { user: { name, age } } = data;

// Rename
const { id: userId } = user;
```

FETCH (GET / POST)

```
// GET
fetch('/api/data')
  .then(r => r.json())
  .then(data => { /* use data */ })
  .catch(err => console.error(err));
// POST
fetch('/api/data', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json'},
  body: JSON.stringify({ name: 'JS' })
})
  .then(r => r.json())
  .then(res => { /* use res */ })
  .catch(err => console.error(err));
```

DOM QUERIES

```
document.getElementById('id')
document.querySelector('css')
document.querySelectorAll('css')
document.getElementsByClassName('c')
document.getElementsByTagName('t')
```

```
// Mini examples
$('*btn')                    // id
document.querySelector('.item')  // first
document.querySelectorAll('li')  // NodeList
```

THIS CHEATS

```
// Regular
function show() { console.log(this); }
// Method
const obj = {
  name: 'JS',
  hi() { console.log(this.name); }
};
obj.hi(); // 'JS'
// Arrow (lexical this)
const obj2 = {
  name: 'JS',
  hi: () => console.log(this)
};
```

FALSY LIST

```
false
0
-0
0n
'' (empty string)
null
undefined
NaN
```

Everything else is truthy!

MODULE (IMPORT / EXPORT)

```
// Named Export
export const sum = (a, b) => a + b;
export const PI = 3.14159;

// Default Export
const greet = (name) => `Hi, ${name}!`;
export default greet;

// Import
import greet, { sum, PI } from './math.js';
import * as utils from './utils.js';
```

QUICK REFERENCES

OPTIONAL CHAINING (?.)

```
obj?.prop
arr?.[0]
fn?.(arg)
obj?.a?.b?.c

// short-circuits on
// null or undefined
```

NULLISH COALESCING (??)

```
const val = data ?? 'default';

count = count ?? 0;

user.name = user.name ?? 'Anon';

// uses right side only for
// null or undefined
```

TEMPLATE LITERALS

```
const name = 'JS';
const msg = `Hello, ${name}!`
Today is ${new Date().toDateString()}`;

// multi-line
const html = `
  <div>
    <h1>${title}</h1>
  </div>`;
```

TERNARY OPERATOR

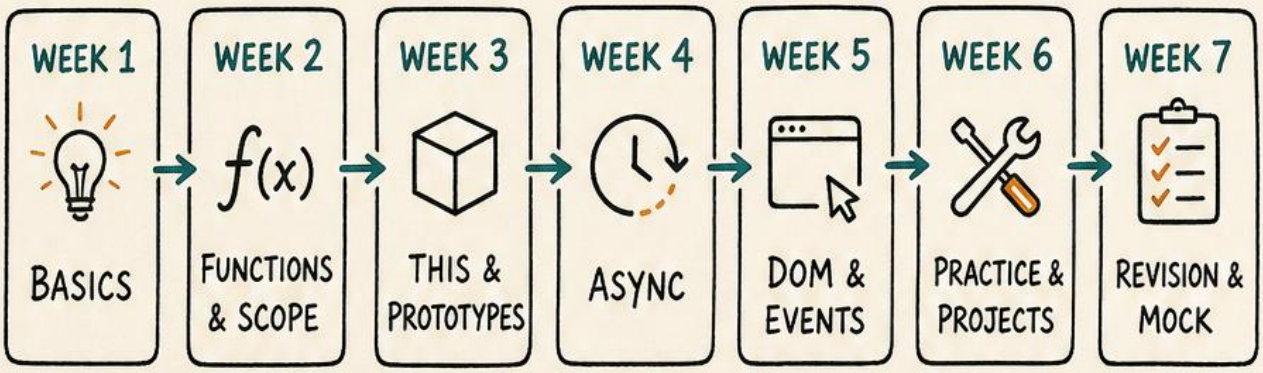
```
const role = isAdmin ? 'Admin' : 'User';
const status = ok ? 'Success' : 'Fail';
const sign = n >= 0 ? 'Positive' : 'Negative';

// quick if-else
// in one line
```

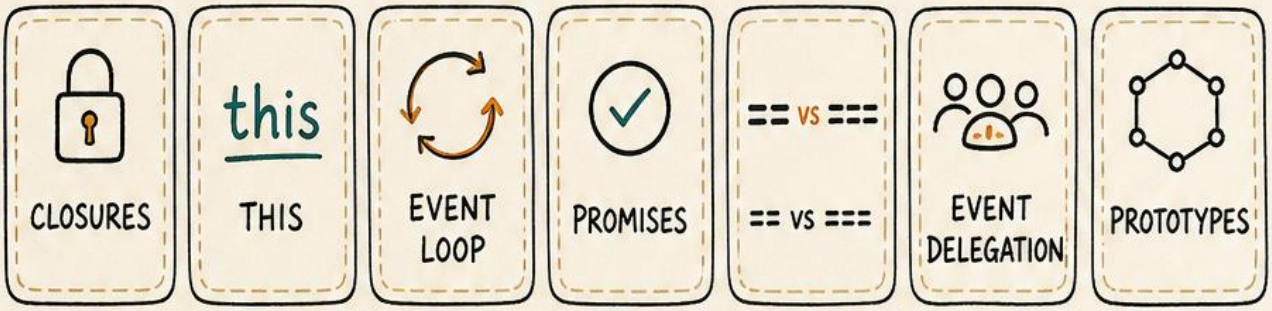
★ Keep this page beside the editor. ★

19 – ROADMAP & MASTER RECAP

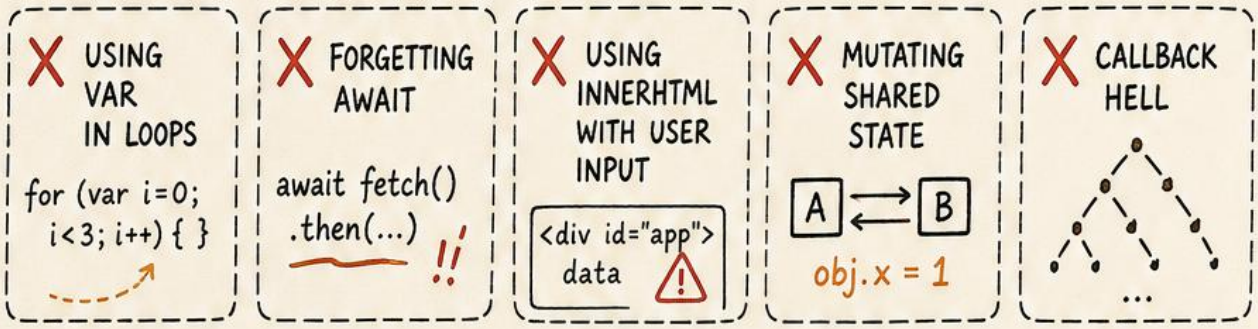
LEARNING PATH – week-by-week –



MUST-KNOW INTERVIEW



COMMON MISTAKES



SCOPE. THIS. EVENT LOOP. ASYNC.